

MODULE 04

工具系统、执行编排与 Checkpoint

从 tool schema 到受控副作用：registry、并发、shell、文件、web、git 与回滚

工具的价值不只在“能做什么”，还在它是否可组合、可取消、可截断、可观察，并能把错误以模型可理解的方式重新注入循环。

研究基线：Claude Code v2.1.236（稳定版） | Python Agent SDK v0.2.141（tag） | 历史源码观察 v2.1.88
资料核验日期：2026-08-19 | 中文深度研究版

目录

- 1. Tool pool 的五步组装
- 2. 内置工具按能力分类
- 3. 文件工具：小 API，强不变量
- 4. Shell 工具：最强能力也是最大攻击面
- 5. 工具编排：并发不是 `Promise.all`
- 6. 输出整形与大结果治理
- 7. Checkpoint：文件可逆，不等于世界可逆
- 8. Git 与 worktree
- 9. Tool 设计检查表

阅读提示：内部模块名以历史源码观察解释，当前功能以官方文档与 v0.2.141 SDK 源码校验。

1. Tool pool 的五步组装



- 从 built-in registry 取得工具定义与 prompt/schema。
- 按平台、入口、permission mode、plan/coordinator 状态过滤。
- 连接 MCP、plugins、LSP，并决定 eager 或 deferred loading。
- 添加 Skill/Agent/Task 等元工具；它们改变的是执行结构，不只是多一个 API。
- 应用 bare-name deny/**tools** availability，最终工具目录进入模型上下文。 [O3, O24, S5, R1]

availability 与 permission 是两层：工具是否出现在模型上下文，与模型调用后是否获准执行，不是同一件事。SDK **tools**/bare disallowed 可移除工具；**allowed_tools**/scoped rules 主要影响批准。 [O3, S5]

2. 内置工具按能力分类

类别	代表工具	副作用	关键约束
发现	Read / Grep / Glob / LSP	低	路径边界、截断、gitignore
编辑	Edit / Write / NotebookEdit	本地文件	旧文本唯一性、checkpoint、编码
执行	Bash / PowerShell	任意子进程	permission、sandbox、timeout
网络	WebFetch / WebSearch	外部读	域名规则、会话限额、prompt injection
编排	Agent / Task* / TodoWrite	上下文与任务状态	隔离、生命周期、成本
扩展	Skill / MCP / LSP	动态能力	供应链、schema、加载成本
交互	AskUserQuestion / ExitPlanMode	人类决策	不能伪造批准

3. 文件工具：小 API，强不变量

- Read：偏移/长度、图片/PDF、行号与截断提示必须明确；读取 deny 还要覆盖 Grep/Glob 的最佳努力路径。
- Edit：用 old_string/new_string 做局部替换，默认要求唯一命中；先读后改降低 stale edit。
- Write：覆盖前确认父目录与现有文件；新文件和覆盖要产生可恢复 snapshot。
- NotebookEdit：按 cell 语义修改，不能把 ipynb 当普通 JSON 文本随意替换。
- LSP：提供定义、引用、诊断等结构化信息；结果仍可能受 gitignore 与 server 状态影响。 [O3, O10]

v2.1.236 前后的 bug 修复显示 Unicode、symlink、worktree、gitignored references 和 CJK 都是实战边界。文件工具正确性来自规范化路径、原子写、编码保真、读取后版本检查，而不是模型“更小心”。 [O1]

4. Shell 工具：最强能力也是最大攻击面

Bash/PowerShell 能调用任意编译器、包管理器、git、容器和云 CLI，因此必须经过命令结构解析、权限规则、路径约束与 OS sandbox。只靠字符串 `startsWith` 无法处理管道、重定向、命令替换、别名和 shell 函数。 [O3, O5, O6, R2]

环节	目的	失败时
命令解析	识别子命令、重定向和组合符	转人工或拒绝
只读判定	允许安全查询免打扰	进入标准 permission
规则匹配	deny/ask/allow	最严格优先
sandbox 构造	FS/network OS 边界	提示或 fail closed
process execution	timeout、stdout/stderr、cwd	结构化 tool error
result shaping	截断与摘要	保留完整日志路径

共享故障模式

历史源码研究发现，复杂复合命令的安全解析可能因性能上限而降级。通用教训是：安全层若共享同一事件循环和超时预算，层数再多也可能同时失效。应把解析预算、fallback 行为和 fail-open/fail-closed 明确化。 [R1, R2]

5. 工具编排：并发不是 `Promise.all`

情况	调度
多个独立 Read/Grep/Glob	并发，结果按 tool_use_id 对齐
同一文件的多个 Edit	串行，后一个基于前一个结果
Bash 生成文件 + Read	依赖边，生成完成后读取
外部部署 + 查询状态	提交后确认 operation id，再轮询/事件等待
用户 interrupt	取消未开始任务；传播 signal；核验已提交副作用

历史实现把 `isConcurrencySafe` 下放给工具定义。这是可扩展设计：orchestrator 不需要硬编码每个工具语义，但工具作者必须承担正确声明的责任。错误地把写工具标为安全会制造难以复现的竞态。 [R2]

6. 输出整形与大结果治理

- 给每条消息与单次工具结果设硬上限，防止日志/二进制填满 context。
- 保留开头、结尾和错误附近，而不是机械截前 N 字符。
- 完整结果写临时文件，模型只看摘要与可继续读取的路径。
- MCP 大结果与 shell 大结果都要有一致的截断提示，不能静默丢失。
- 错误输出要保留 exit code、command、cwd、timeout/kill 原因。

7. Checkpoint：文件可逆，不等于世界可逆

Claude Code 在每个用户 prompt 前建立恢复点，并在文件编辑前保存内容。`/rewind` 可恢复 conversation、code 或两者；checkpoint 独立于 git，resume 后仍可用。SDK 需显式 `enable_file_checkpointing`，再以 user message UUID 调 `rewind_files()`。 [O10, S2, S5]

能恢复	不能恢复/有限恢复
Edit/Write 创建或修改的普通文件	Bash 内部产生的任意修改可能不完整追踪
对应 prompt 之前的内容 snapshot	数据库、API、部署、发消息等远端副作用
conversation 与 code 可分别选择	symlink/hard link/父目录解析变化会跳过
会话内历史 checkpoint	不是 git 分支，也不替代 commit

最佳实践

在执行不可逆外部动作前，把批准点设计成显式事务边界：展示目标、diff/plan、dry-run 结果和回滚方案。Checkpoint 只能兜底文件编辑，不能撤销生产发布。

8. Git 与 worktree

- 读操作加 `--no-optional-locks` 可减少文件 watcher 循环和无谓锁。
- worktree 的 `.git` 是指针，远端与主配置可能在 common dir；后台任务不能只读 worktree gitdir。
- 并行 agent 最安全的写隔离是每个任务独立 worktree；共享仓库只能把工具限制为只读。
- checkpoint 与 git 各有职责：前者细粒度会话回退，后者可审查、可分享的工程历史。
- 任何自动 commit/push/PR 都应分成独立授权步骤。 [O1, O10]

9. Tool 设计检查表

- Schema 是否足够窄、字段名是否能让模型正确选择？
- 有没有 `tool_use_id`、timeout、abort 与幂等键？
- 输入是否规范化并在执行前再次验证？
- 输出是否有大小上限、错误结构与可追溯原文？
- 是否声明并发安全、可逆性和所需权限？
- Pre/Post hooks 能否观察且不会改变核心语义？
- 恢复/重试会不会重复产生外部副作用？

参考资料与证据等级

[O3] 官方 | [Tools reference](#)

<https://code.claude.com/docs/en/tools-reference>

[O10] 官方 | [Checkpointing](#)

<https://code.claude.com/docs/en/checkpointing>

[O24] 官方 | [Agent SDK agent loop](#)

<https://code.claude.com/docs/en/agent-sdk/agent-loop>

[S2] 开源源码 | [Internal Query control protocol](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/query.py

[S3] 开源源码 | [Subprocess CLI transport](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/transport/subprocess_cli.py

[S4] 开源源码 | [Message parser](#)

https://github.com/anthropics/claude-agent-sdk-python/blob/v0.2.141/src/claude_agent_sdk/_internal/message_parser.py

[R1] 社区研究 | [Dive into Claude Code v2.1.88](#)

<https://github.com/VILA-Lab/Dive-into-Claude-Code>

[R2] 社区研究 | [Claude Code Research - source analysis](#)

<https://github.com/cablate/claude-code-research>

方法说明

官方材料用于确认当前稳定行为；SDK 源码固定在发布 tag v0.2.141；社区材料只用于解释 v2.1.88 历史源码结构。源码行号和内部函数名可能在后续版本变化，外部契约以官方最新文档为准。